Resúmenes de papers.

1. Benchmarking Large Language Models on Repairing Qiskit Programs using Bugs4Q

Motivación y Problema Principal

- **Inestabilidad de los benchmarks cuánticos:** Benchmarks como *Bugs4Q* sufren una "inversión silenciosa de etiquetas". Al actualizarse Qiskit, los programas con fallos dejan de reproducir el error o las correcciones de referencia dejan de ejecutarse sin emitir mensajes de error evidentes.
- **Dependencia del entorno:** La validez real del benchmark depende de la combinación exacta del par (caso de prueba, versión de Qiskit) y no del benchmark por sí solo.

Metodología

- **Construcción del benchmark:** Se combinaron dos conjuntos de *Bugs4Q* (*Bugs4Q-Framework* y *Bugs4Q-NA*), consolidando 67 defectos reales de Qiskit deduplicados.
- **Tests ejecutables:** Se escribieron y revisaron oráculos ejecutables basados en el módulo bajo prueba (MUT).
- **Evaluación fijada por versión:** Cada caso se validó bajo 6 versiones distintas de Qiskit: 0.25.0, 0.45.0, 1.0.0, 1.1.1, 2.0.0 y 2.3.1.
- **Modelos evaluados:** GPT-4o-mini, GPT-5o-mini, GPT-5.4 y GPT-5.4-mini, evaluando hasta 10 candidatos de reparación por defecto.

Resultados Clave

- **Rendimiento General (Pass@10 sin filtrar por validez):**
    - **GPT-5.4:** 48.8%
    - **GPT-5.4-mini:** 47.3%
    - **GPT-5o-mini:** 30.3%
    - **GPT-4o-mini:** 22.6%

- **Impacto Drástico de la Versión:** Todos los modelos rinden sustancialmente mejor en Qiskit 0.45.0 y sufren caídas drásticas de desempeño a partir de Qiskit 1.0 y las familias 2.x.
- **Incompatibilidad de Entorno vs. Lógica:** Un porcentaje significativo de fallas (entre el 13% y el 56%, según el modelo) se debe únicamente a APIs obsoletas o incompatibles. Además, el **64% de las reparaciones exitosas** ocurrió en entradas que habían sido invalidadas en la versión de prueba.

Conclusión y Recomendaciones

- Evaluar el APR cuántico con métricas genéricas y agnósticas de la versión distorsiona gravemente los resultados reales.
- Es fundamental adoptar protocolos de evaluación con **versiones fijas del entorno** y un seguimiento explícito de la ejecutabilidad del código.
- Los autores liberaron el conjunto de datos revalidado y fijado por versión en su repositorio público de GitHub.

2. On Repairing Quantum Programs Using ChatGPT

Contexto y motivación

- **El desafío cuántico:** Superposición, entrelazamiento y ruido en dispositivos NISQ generan errores específicos (puertas, mediciones, etc.) difíciles de detectar y corregir a mano.
- **La brecha en APR:** La reparación automática clásica con LLMs avanzó mucho, pero en computación cuántica faltan técnicas automatizadas dedicadas.

Metodología y diseño experimental

- **Benchmark:** 38 errores reales y reproducibles de Bugs4Q, en Qiskit.
- **Cuatro enfoques de prompt:**
    - **Chat:** Solo código con fallos.
    - **Chat_D:** Código + descripción básica.

    - **Chat_2D:** Código + descripción detallada con intención y comportamiento esperado.
    - **Chat_3D:** Igual que Chat_2D, pero informando que el programa contiene errores sin preguntar si hay un bug.
- **Diálogo iterativo:** Si el primer parche fallaba al ejecutarse o no pasaba las pruebas, se daba a ChatGPT el mensaje de error o la información del test fallido.
- **Métricas:** TOP-5 (al menos uno de 5 intentos corrige el error) y AVG-5 (tasa de éxito promedio entre las 5 iteraciones).

Resultados principales

- **Eficacia general:** ChatGPT reparó 29 de 38 errores (76,3 % en TOP-5) con descripciones detalladas y retroalimentación de las pruebas.
- **Nivel de detalle:** Solo código (Chat) reparó 10 de 38; con información detallada (Chat_2D y Chat_3D) reparó 29 de 38.
- **Por tipo de error (Chat_3D):**
    - **Errores clásicos:** 100 % TOP-5 (6/6) y 97 % AVG-5.
    - **Cuánticos simples:** 84 % TOP-5 (21/25) y 67 % AVG-5.
    - **Algoritmos cuánticos complejos:** 28,5 % TOP-5 (2/7) y 17 % AVG-5.
    - *Nota:* Falló sobre todo en algoritmos propios o personalizados ajenos a su corpus de entrenamiento público.

Conclusiones y discusión

- **Factibilidad:** Es factible usar LLMs como ChatGPT para asistir en la reparación de código cuántico.
- **Herramienta semiautomática:** La efectividad depende de la capacidad del desarrollador para describir el error y dar contexto en el prompt.
- **Limitaciones:** Deficiencias en errores muy complejos y en algoritmos cuánticos novedosos no vistos en el entrenamiento.

Aquí tienes exactamente el mismo texto con un formato estructurado y profesional en Markdown, listo para seleccionar, copiar y pegar directamente en Microsoft Word:

3. A Unified Multi-Framework Benchmark for LLM-Based Quantum Code Generation (QuanBench+)

Contexto y problema

- **Aislamiento de habilidades:** La mayoría de benchmarks evalúan un solo framework (p. ej. Qiskit), lo que impide distinguir razonamiento cuántico conceptual de familiaridad con la API.
- **Naturaleza probabilística:** El código cuántico produce distribuciones de probabilidad, lo que exige métricas de evaluación adecuadas.

La propuesta: QuanBench+

- **Estructura unificada:** 42 tareas funcionales que se ejecutan y evalúan de forma idéntica en Qiskit, PennyLane y Cirq.
- **Categorías:** Algoritmos cuánticos (31 tareas), preparación de estados (6) y descomposición de puertas (5).
- **Criterio de corrección:** Ejecución directa del código; divergencia KL con umbral 0,05 frente a la solución canónica; se excluye la fidelidad de estado/unitaria porque da falsos negativos en circuitos optimizados o transpilados funcionalmente equivalentes.

Preguntas de investigación y resultados

- **RQ1, Pass@1:** Qiskit es el objetivo más fácil, Cirq intermedio y PennyLane el más difícil.
    - *Máximos:* Qiskit 59,5 % (Gemini 3 Pro), Cirq 54,8 % (Gemini 3 Pro), PennyLane 42,9 % (GPT-5.1). La generación fiable entre frameworks sigue sin resolverse.
- **RQ2, prefill:** Incluir importaciones y firmas reduce errores de sintaxis y ayuda sobre todo a modelos medianos o pequeños, pero no resuelve fallos profundos de lógica.
- **RQ3, reparación por retroalimentación (Pass@1 FB):** Hasta 5 intentos cuando la ejecución falla.

    - *Resultados:* Qiskit sube a 83,3 % (GPT-5.1), Cirq a 76,2 % (Gemini 3 Pro) y PennyLane a 66,7 % (GPT-5.1 y Gemini 3 Pro).

- **Análisis de errores:** Los errores de sintaxis caen de 4,7 % a 1,5 %. Los fallos residuales están dominados por respuestas incorrectas (53,4 %) y errores de lógica (22,0 %).

Conclusiones

Las capacidades de los LLMs para programar software cuántico dependen más de la exposición a las APIs de un framework (mayor en Qiskit) que de un razonamiento cuántico transferible. La retroalimentación en tiempo de ejecución mejora mucho la tasa de éxito, pero superar el rendimiento actual requerirá mejor entrenamiento en razonamiento composicional cuántico.

4. Explainable Quantum Program Repair with Verifiable Proof Traces

Problema y motivación

- **Limitación de los LLMs:** Sus explicaciones en lenguaje natural son informales, posteriores a la generación y no comprobables.

- **Sensibilidad cuántica:** Pequeñas variaciones sintácticas pueden cambiar la semántica del circuito (equivalencia de puertas, fidelidad), y el texto no garantiza corrección semántica.

- **Solución propuesta:** Tratar las explicaciones como objetos ejecutables y verificables mediante trazas de prueba (*proof traces*).

Metodología (tres etapas)

- **Generación de candidatos y justificaciones estructuradas:** Dado un programa defectuoso (QASM, Qiskit o Cirq) y una especificación, el LLM propone parches con justificaciones estructuradas (reordenamiento de puertas, sustituciones).

- **Compilación a trazas de prueba y verificación, con backends formales:** Equivalencia de circuitos, ZX-calculus, análisis de estabilizadores (circuitos de Clifford) y simulación cuántica (fidelidad).

- **Selección de reparaciones fieles:** Solo se aceptan soluciones cuyas explicaciones están 100 % verificadas.

Evaluación experimental

- **Benchmark:** 500 circuitos de QASMBench (2 a 130 qubits).
- **Mutaciones:** 5.000 circuitos con errores, con 5 tipos de mutación (eliminación/inserción de puertas, reordenamiento, perturbación de parámetros y alteración de índices de qubits).
- **Éxito de reparación:** 76,1 %, por encima de LLM-QAPR (72,9 %) y HornBro (74,6 %).
- **Precisión semántica:** 96,1 %.
- **Fidelidad de la explicación:** 94,8 % de explicaciones verificadas, frente a 18,6 % en explicaciones en lenguaje natural tradicionales.
- **Distancia de edición:** Promedio más bajo (0,28), con cambios mínimos.

Conclusiones clave

- Exigir explicaciones verificables no reduce el éxito si se generan y filtran múltiples candidatos, y mejora mucho la precisión semántica.
- ZX-calculus fue el motor de verificación más crítico.
- Basar las explicaciones en semántica ejecutable y verificable es clave para herramientas de IA fiables en software cuántico.

5. Large Language Models for Quantum Software Engineering: Opportunities and Challenges in Modeling, Generation, Testing, and Repair

Las 4 áreas de aplicación

- **A. Modelado y análisis:** Resumir la intención y semántica de circuitos y patrones de errores, búsqueda semántica de código, generación de modelos formales (p. ej. grafos de dependencia cuántica) y traducción entre SDKs (Qiskit, Cirq, Q#).

- **B. Generación de código:** Traducir descripciones en lenguaje natural a código ejecutable, diseñar estructuras de circuitos parametrizados (PQC) para VQE o QAOA y coordinar flujos híbridos cuántico-clásicos.
- **C. Pruebas:** Generar casos de prueba inteligentes (casos límite), asistir en oráculos y aserciones e identificar patrones de error propios del dominio cuántico.
- **D. Reparación automática (APR):** Generar parches a partir de registros de ejecución, coordinar flujos de reparación de extremo a extremo y asistir reparaciones conscientes del hardware.

Desafíos únicos del dominio cuántico

- **Escasez de datos de entrenamiento de calidad:** Faltan repositorios grandes de código cuántico etiquetado o pares error-corrección.
- **Brecha de corrección semántica y alucinaciones:** Código sintácticamente correcto puede tener errores semánticos que alteran la evolución del estado.
- **Problema del oráculo de prueba:** Al ser probabilísticos los resultados, verificar un parche exige inferencia estadística sobre miles de ejecuciones (*shots*).
- **Brecha simulación-hardware:** Lo validado en simuladores ideales suele fallar en procesadores NISQ reales por decoherencia, errores de puertas y conectividad limitada.
- **Alineación con restricciones físicas:** Optimizar exige reducir profundidad y puertas CNOT y adaptarse a la topología del chip.

Hoja de ruta hacia 2030

Corto plazo (2025–2026), problemas fundamentales:

- Síntesis y aumentación de datos y benchmarks unificados.
- Integración con verificación formal y prompts tipo *Chain-of-Thought* orientados a física cuántica.
- Pruebas metamórficas, basadas en propiedades y diferenciales para el problema del oráculo.

Mediano plazo (2027–2028), practicidad y escalabilidad:

- Topologías físicas y calibración de hardware en el prompt/modelo.

- Reducción de la brecha simulación-hardware con reparación consciente del ruido y decodificación de corrección de errores asistida por LLM.
- Detección estática y dinámica de vulnerabilidades en representaciones intermedias (QIR).

Largo plazo (2029–2030), colaboración humano-IA:

- Interfaces con humano en el bucle para descubrir algoritmos.
- Explicabilidad profunda del diseño de circuitos.
- Métricas de cobertura cuántica escalables para propiedades como el entrelazamiento.

6. Beyond Rules: LLM-Powered Linting for Quantum Programs

Motivación y problema

- **Inadecuación de las reglas fijas:** Los linters deterministas (como LintQ) requieren mucho mantenimiento manual y se adaptan mal a la evolución de las APIs y a problemas dependientes del contexto.
- **LintQ-LLM+CoT:** Secuencia estructurada de *Chain-of-Thought*: planificación estratégica, comprensión del código, lógica de detección e informe estructurado.
- **LintQ-LLM+RAG:** Añade recuperación (RAG) sobre una base de conocimiento vectorial (FAISS) de casos reales verificados.

Metodología

- **Categorías:** 10 tipos de problemas cuánticos (p. ej. OpAfterMeas, InsuffClasReg, OldIdenGate).
- **Corpus:** 55 programas Qiskit (43 reales y 12 sintéticos con fallos inyectados para categorías raras).
- **Métricas:** Precisión, recall y F1 frente a un *ground truth* validado manualmente y al linter LintQ.

Resultados

- **RQ1:**
    - *LintQ:* F1 0,41 (precisión 0,39; recall 0,43).
    - *LintQ-LLM+CoT:* F1 0,70 (precisión 0,55; recall 0,96).
    - *LintQ-LLM+RAG:* F1 0,68 (precisión 0,56; recall 0,86).
- **RQ2, impacto de RAG:** Compromiso entre precisión y sensibilidad. RAG reduce falsos positivos (0,56 vs 0,55) pero baja el recall (0,86 vs 0,96) al depender de plantillas conocidas. RAG conviene para evitar falsas alarmas; CoT puro para no dejar escapar errores.
- **Resistencia a la ofuscación:** El modelo razona sobre propiedades topológicas del circuito, no sobre nombres de variables memorizados.
- **Calidad vs. cantidad en RAG:** La efectividad depende de la claridad sintáctica y estructural del tipo de error, no del volumen de ejemplos.

Conclusión

Los LLMs con CoT y RAG dan una base adaptable y escalable para el aseguramiento de calidad de software cuántico, trasladando el esfuerzo de escribir reglas a la ingeniería de prompts y la curación de bases de conocimiento.

7. Towards Quantum Program Bug Datasets and Benchmarking (QBench)

Motivación y problema

- **Ausencia de benchmarks reproducibles:** Faltaba un benchmark estandarizado y ejecutable para evaluar la reparación de código cuántico.
- **Límites de las métricas clásicas:** Resultados estocásticos; equivalencia con tolerancia numérica; fragilidad del entorno de ejecución (versiones de compiladores, simuladores y bibliotecas); defectos multicapa entre compiladores, simuladores y código de aplicación.

La propuesta: QBench

520 tareas de reparación reales de 129 repositorios de software cuántico. Cada instancia empaqueta un entorno aislado, un oráculo de corrección y el protocolo de evaluación. Tiene dos pistas:

- **RepoTrack (264 instancias):** Errores de infraestructura (compiladores, simuladores, bibliotecas base); suele requerir cambios en varios archivos y razonamiento a nivel de repositorio.
- **ScriptTrack (256 instancias):** Programas de usuario y flujos experimentales (algoritmos, scripts de prueba, entrenamiento variacional); parches más localizados con fuerte lógica de dominio.
- **Oráculos soportados:**
    - *Deterministas:* Pruebas unitarias, excepciones, contratos de API.
    - *Con tolerancia numérica:* Equivalencia aproximada por umbrales.
    - *Probabilísticos:* Evaluación estadística con *shots* sobre distribuciones de medición.

Resultados de la evaluación empírica (Pass@1 global)

- **LLMs frontera generales (GPT-4o, Claude-3.5-Sonnet, Gemini-2.5-Pro):** Hasta 50,1 % global (38,6 % en RepoTrack y 61,7 % en ScriptTrack).
- **LLMs especializados en código (DeepSeek-Coder, Qwen2.5-Coder, Codestral-22B):** Entre 34,4 % y 36,5 %.
- **LLMs de pesos abiertos (LLaMA-3.1, DeepSeek-R1):** Entre 29,1 % y 39,7 %.
- **Baselines tradicionales (plantillas heurísticas y APR basado en búsqueda):** Entre 10,9 % y 17,1 %.
- **Por tipo de oráculo:** Mejor rendimiento con oráculos deterministas (más de 60 % en modelos frontera); cae con oráculos probabilísticos (menos de 25 % en RepoTrack), pues los modelos generan parches frágiles que pasan una ejecución pero fallan sobre distribuciones repetidas.
- **Retroalimentación del oráculo:** Permitir errores y reintentos (hasta $B = 10$ iteraciones) casi duplicó el éxito en RepoTrack, al corregir errores de entorno, compilación y dependencias.

Conclusiones

La naturaleza estocástica de los oráculos, la fragilidad del entorno y los defectos entre capas son los mayores obstáculos para la reparación automática de código cuántico. El benchmark se liberó públicamente.

8. Generic or Quantum-Specific? Why the Control Matters: Difficulty-Matched Probes of LLM Quantum-Code Failures

Problema y aportación metodológica

- **Control inadecuado:** Muchas evaluaciones suponen que los fallos son cuánticos por naturaleza sin probarlo, usando un control estructural clásico (misma forma o tamaño de salida).
- **Hallazgo central:** Con un control estructural simple casi todos los fallos parecen cuánticos, pero ese control es tan fácil que los modelos lo aprueban el 99,5 % de las veces, sobreestimando la especificidad cuántica.
- **Control ajustado por dificultad:** Se aparea la FFT clásica con la QFT de 3 qubits (QFT-3). Ambas comparten la dificultad de componer en orden operaciones no conmutativas, pero la FFT no requiere física cuántica.

Diseño experimental

- **Modelos y evaluadores:** 9 modelos de 4 proveedores (entre ellos Claude Opus 4.7/Sonnet 4.6/Haiku 4.5, GPT-5.5/GPT-5.4-Mini, Gemini 2.5 Flash, Qwen2.5-Coder-32B y Llama-3.3-70B), con 12 prompts apareados en 5 niveles de dificultad.
- **Evaluación:** Por simulación directa en Qiskit y clasificación con una taxonomía de tipos de fallo.

Resultados principales

- **Especificidad cuántica (QFT-3):** Con control estructural simple, 100 % de los fallos parecían cuánticos; con control ajustado por dificultad (FFT) cae al 50 % (8 de 16).
- **Según capacidad del modelo:** Los avanzados (GPT-5.4-Mini, Claude Haiku 4.5) aprueban la FFT pero fallan en QFT-3, un déficit realmente cuántico. Los débiles (Gemini 2.5 Flash-Lite, LLaMA-3.3-70B) fallan en ambas, una incapacidad genérica para ordenar operaciones complejas.
- **Inconmensurabilidad en tareas nativas cuánticas:** En 44 de 63 fallos cognitivos (estado de Bell, oráculo Toffoli, etc.) el objetivo clásico es trivial (un

numpy.array) y toda la dificultad está en las puertas cuánticas. Sin equivalente clásico complejo no se puede medir la especificidad, y el autor declara no atribuirlos como fallos cuánticos directos.

- **Patrón predominante:** «Plantillas visuales sin verificación semántica». Los LLMs reproducen la apariencia de un circuito (p. ej. Hadamard seguidas de fases controladas) sin verificar que la unitaria resultante calcule la transformación pedida.
- **Retroalimentación:** Los errores de orden de puertas (QFT-3) son muy recuperables con feedback del simulador; los de compromiso con una receta errónea (oráculos de fase) persisten tras varios intentos.

Conclusiones y recomendaciones

- **Atribución de fallos:** Afirmar que un LLM falla por «física o lógica cuántica» exige comparar con controles clásicos de igual dificultad algorítmica.
- **Límite de los modelos especializados:** Qwen2.5-Coder-32B siguió fallando frente a modelos generales de frontera (Claude Sonnet 4.6, GPT-5.5, con 100 % de éxito); la especialización en código clásico no sustituye el razonamiento abstracto a mayor escala.

Aquí tienes exactamente el mismo texto adaptado al formato estructurado y profesional en Markdown, listo para copiar y pegar directamente en Microsoft Word:

9. Are Large Language Models Ready for Quantum Software Engineering?

RQ1: Cobertura del ciclo de vida de QSE

La adopción de LLMs se centra en el código y descuida las fases iniciales y finales:

- **Implementación (fase dominante):** Generación de circuitos, código Qiskit/ PennyLane y asistencia interactiva.
- **Análisis y mantenimiento:** Reparación de código, refactorización y migración entre versiones de SDKs, comprensión de algoritmos y linting.
- **Pruebas y diseño:** Muy poco explorados (un par de estudios sobre *fuzzing*, pruebas *flaky* y diseño de *ansatz*).
- **Fases ausentes:** Sin evidencia en ingeniería de requisitos, despliegue o mantenimiento avanzado de infraestructura.

RQ2: Modelos más usados

- **Dominio de modelos propietarios:** La familia GPT de OpenAI (GPT-3.5, GPT-4, GPT-4o, GPT-4.1) domina, ante la complejidad y escasez de datos en programación cuántica.
- **Alternativas abiertas:** Uso emergente pero fragmentado de LLaMA, DeepSeek, StarCoder, Granite, Qwen y Mistral, a menudo como complemento.
- **Riesgo metodológico:** Depender de modelos cerrados y cambiantes compromete la reproducibilidad y la comparación longitudinal.

RQ3: Limitaciones y desafíos (8 temas; el documento enumera 6)

- **Inexactitud semántica y alucinaciones:** Circuitos válidos en sintaxis pero lógicamente incorrectos o inejecutables en procesadores reales.
- **Escasez y cobertura de datos:** Sobre APIs recientes, conceptos físicos y patrones avanzados.
- **Degradación con la complejidad:** Bien en tareas simples o de libro de texto, mal en problemas reales complejos.
- **Alta sensibilidad al prompting.**
- **Problemas de alineación:** Y explicaciones superficiales que no reflejan la intención del desarrollador.
- **Limitaciones de evaluación:** La mayoría de estudios usan benchmarks simplificados o restrictivos.

Conclusión del estudio

Los LLMs no están listos para actuar como agentes autónomos en todo el ciclo de vida de QSE; hoy son herramientas de asistencia limitada en tareas acotadas de código. El software generado por LLMs debe tratarse como artefacto provisional y pasar por puertas de validación estrictas (compilación, simulación y ejecución en backends) antes de integrarse.

10. How AI, LLMs and Quantum Science Can Empower Each Other?

1. Introducción y fundamentos de los LLM

- **IA generativa:** Redes neuronales profundas (arquitectura Transformer, 2017) entrenadas con grandes volúmenes de datos para generar contenido nuevo.
- **Tokens y embeddings:** Los LLM convierten el texto en tokens representados como vectores en espacios multidimensionales para predecir la continuación del texto.
- **Razonamiento:** No tienen razonamiento deductivo o inductivo genérico; su «razonamiento» procede de patrones estáticos de los datos de entrenamiento.
- **Chain of Thought / Tree of Thoughts:** Guían al modelo por pasos intermedios para descomponer problemas.
- **Fine-tuning e instruction tuning:** Ajustan pesos (p. ej. LoRA) o el seguimiento de instrucciones para dominios concretos.
- **RLHF / DPO:** Alinean las respuestas con preferencias y reglas éticas humanas.
- **RAG:** Conecta el LLM con bases vectoriales externas (p. ej. arXiv) para información actualizada, menos alucinaciones y fuentes citables sin reentrenar.
- **Propuesta:** Un chatbot basado en un LLM de código abierto especializado en ciencia cuántica, alimentado vía RAG con el corpus científico del sector (p. ej. todo arXiv cuántico).

2. Aplicaciones de los LLM en la ciencia cuántica

Se analiza el impacto de GPT-4 y Google Bard/Gemini (finales de 2023 / principios de 2024):

- **A. Aprendizaje y estudio:** Explican con precisión superposición, entrelazamiento y desigualdades de Bell, pero fallan en razonamientos específicos o reglas físicas sutiles (Bard «alucinó» si el Rubidio-87 es bosón o fermión tras la presión del usuario). GPT-4 sacó 69/100 en el examen final de Scott Aaronson, pero falla en preguntas abiertas y cálculos analíticos estrictos.
- **B. Revisión de artículos:** Resumen rápido de *papers* de arXiv, extracción de puntos clave, verificación de estructuras formales y evaluación de metodologías; los visores de IA ayudan a revisar coherencia, bibliografía y sesgos.
- **C. Redacción y gestión científica:** Apoyo en solicitudes de financiación, resúmenes, textos para políticas nacionales de cuántica (NQI) y cartas de presentación. Se advierte contra su uso坑 indiscriminado en estudiantes: merma

el pensamiento crítico, incentiva el «meta-plagio» y propaga ideas infundadas (*hype*).

- **D. Desarrollo de software:** Muy efectivos para escribir y optimizar Python (Qiskit, Cirq), expresiones regulares, scripts de datos y gráficos. Existen Qiskit Code Assistant (IBM) y Azure Quantum Copilot (Microsoft). Inventan llamadas a APIs inexistentes cuando la lógica algorítmica es compleja y poco presente en repositorios públicos.
- **E. Ideación e investigación colaborativa:** Actúan como «máquinas de hipótesis»; no generan ciencia nueva por sí solos pero hibridan conceptos de varias disciplinas (p. ej. mitigar ruido ambiental en qubits superconductores, nuevos materiales) y traducen jerga entre físicos e ingenieros.

3. Evaluaciones empíricas (materiales suplementarios)

- **Buen desempeño:** Estructurar hojas de datos y figuras de mérito (fuentes y detectores de fotones individuales, amplificadores paramétricos TWPA, circuladores) y generar código de análisis de datos.
- **Citas falsas:** Google Bard inventó repetidamente referencias inexistentes sobre *bosonic qubits*.
- **Gráficos (DALL-E):** Intento fallido de diagramas del algoritmo de Shor o esquemas de qubits; imágenes artísticas con textos y conexiones sin sentido físico.
- **Tablas comparativas de la industria:** Bard atribuyó fidelidades de puertas de 2 qubits a arquitecturas sin puertas (D-Wave) y a chips de control crio-CMOS (Intel Horse Ridge).

4. Otras tecnologías de Machine Learning en la ciencia cuántica

- **Física y hardware:** Estimación del estado fundamental en sistemas de muchos cuerpos, calibración dinámica de qubits, optimización de pulsos de microondas, enrutado de qubits y mitigación de ruido.
- **Software y herramientas:** Optimización de compiladores y transpiladores con aprendizaje por refuerzo profundo, decodificación de síndromes en QEC con Transformers, caracterización de detectores en sensores cuánticos y optimización de redes QKD.

5. Computación cuántica aplicada a la IA y los LLM

- **QML:** En la era NISQ no ha demostrado ventaja cuántica real frente al cómputo clásico con datos a gran escala.
- **Inviabilidad en LLM:** No pueden procesarse en computadoras cuánticas a corto ni medio plazo: no están diseñadas para *Big Data*, la entrada/salida y carga de estados es muy lenta y haría falta una qRAM funcional que no existe.
- **Algoritmos inspirados en la cuántica:** Clásicos pero basados en matemáticas cuánticas (redes de tensores) sí dan beneficios hoy, p. ej. CompactifAI (Multiverse Computing), que comprime LLMs y reduce su tamaño y huella energética.

Conclusiones principales

- **Impacto asimétrico:** La IA y el ML clásico transforman la investigación, educación y desarrollo cuánticos; que la computación cuántica beneficie a la IA o los LLM es muy incierto a corto y medio plazo.
- **Rol del LLM:** Asistente o amplificador, no sustituto del juicio crítico; el conocimiento previo del usuario es obligatorio para filtrar alucinaciones y sesgos mediante «post-selección».
- **Recomendación:** Dominar la ingeniería de prompts, adoptar flujos basados en RAG y colaborar en herramientas e infraestructuras de IA de código abierto específicas del dominio cuántico.
-

Este trabajo (Baresi et al., 2026) evalúa la capacidad de los Modelos de Lenguaje de Gran Escala (LLM) para generar código de *solvers* (resolutores) cuánticos correctos para problemas científicos complejos.

1. La Metodología: Q-SAGE

Los autores presentan **Q-SAGE** (*Quantum-Solver Automated Generation*), un marco de trabajo basado en un bucle iterativo de **generación, ejecución y verificación numérica**:

- **Generación:** Se le proporciona un prompt al LLM indicando el problema físico/matemático, el stack de bibliotecas exacto (p. ej., Qiskit) y los requerimientos del ejecutable.
- **Ejecución:** El código generado se ejecuta en un entorno controlado.
- **Verificación (Óraculo):** El resultado numérico del código generado se compara con el resultado exacto de un *solver* clásico de referencia (utilizando diagonalización exacta, métodos ab initio, etc.).
- **Refinamiento (Feedback):** Si el código falla o el resultado numérico no coincide dentro de una tolerancia, se le reenvía el error o resultado al LLM para que corrija su código durante múltiples iteraciones (hasta 10 intentos).

2. Experimentos

- **Modelos evaluados:** DeepSeek-V3.2, gpt-oss-120b, Qwen3-Coder-480B, Qwen3-Coder-Next-FP8 y GPT-5.2.
- **Familias de Problemas (PF):**
    1. **PF1 (Modelo Fermi-Hubbard - VQE):** Física de la materia condensada.
    2. **PF2 (Modelo Ising de Campo Transversal - VQE):** Física de muchos cuerpos / sistemas de espín.
    3. **PF3 (MaxCut - Quantum Annealing):** Optimización combinatoria.
    4. **PF4 (Modelo Schwinger - Trotterización):** Teoría de campos en red.
    5. **PF5 (Estructura Electrónica Molecular - QPE):** Química cuántica.

3. Principales Hallazgos y Respuestas a las Preguntas de Investigación

- **Efectividad (RQ1):**

- La retroalimentación iterativa incrementa de forma sustancial las tasas de éxito (por ejemplo, pasando del intento 1 a los intentos 5 o 10).
- La mayoría de las mejoras se logran en los **primeros 5 intentos**.
- La efectividad varía enormemente según la dificultad del problema: problemas con mapeos directos (**PF2 y PF3**) alcanzaron tasas de éxito superiores al 85%, mientras que problemas con transformaciones complejas de fermiones a qubits o dinámica temporal (**PF1, PF4, PF5**) mostraron un rendimiento significativamente más bajo.
- **GPT-5.2** obtuvo el desempeño general más alto.

- **Causas de Falla (RQ2):**
  - A medida que los modelos aumentan en capacidad, sus modos de falla sufren una **transición**:
    - **Modelos menos capaces (o más pequeños):** Fallan principalmente por errores sintácticos, problemas con formateo de respuestas, uso incorrecto de APIs o incompatibilidad de bibliotecas.
    - **Modelos avanzados (como GPT-5.2):** Logran generar código ejecutable sin errores de entorno, pero fallan mayoritariamente debido a **imprecisiones numéricas** (el algoritmo corre, pero da un resultado numéricamente incorrecto).

- **Costo y Sobrecarga Computacional (RQ3):**
  - Aunque el bucle iterativo eleva la tasa de éxito, introduce un **costo computacional y de tiempo considerable**.
  - Obtener la primera solución correcta mediante iteración requiere significativamente más tiempo que un único intento, presentando colas largas en la distribución de tiempo en los problemas más complejos.

4. Conclusión

El estudio demuestra que los LLMs no pueden considerarse "correctos" únicamente porque su código se ejecute sin arrojar excepciones. Para el cómputo científico y cuántico, la métrica crítica es la **precisión numérica**. Si bien el feedback iterativo ayuda a corregir fallas de código tradicionales, la imprecisión numérica sigue siendo la mayor barrera para la automatización del desarrollo de software cuántico mediante LLMs.

Este trabajo (Le, Tran-Truong y Nguyen, 2026) examina cómo el "grounding" (fundamentación mediante técnicas como RAG, análisis estático y retroalimentación de ejecución) afecta la capacidad de los Modelos de Lenguaje de Gran Escala (LLM) para detectar y reparar fallas en programas cuánticos escritos en Qiskit.

1. El Sistema: LINTREPAIR-RAG-Q

Los autores desarrollan una canalización (*pipeline*) de reparación que integra tres componentes:

- **Analizador Estático:** Un linter basado en AST de reglas fijas para detectar errores sintácticos/de estructura (p. ej., llamadas a APIs eliminadas en Qiskit 1.0, índices de qubits fuera de rango o registros mal dimensionados).
- **Detector LLM asistido por RAG:** Consulta una base de conocimientos (KB) con descripciones de patrones de defectos para categorizar problemas.
- **Bucle de Reparación con Validación:** El LLM genera un parche alimentado con los hallazgos del linter y conocimiento recuperado. El parche se ejecuta en un subproceso aislado; si arroja un error de tiempo de ejecución, la excepción se retroalimenta al modelo (hasta 3 intentos).

2. Benchmark y Datos: Q-DEFECTS40

El estudio evalúa la metodología sobre **Q-DEFECTS40**, un benchmark compuesto por 40 programas en Qiskit 2.x (32 defectuosos y 8 limpios), divididos en tres grupos:

1. **Migración de API (12 programas):** Defectos causados por incompatibilidad con Qiskit 1.0+ (provocan fallas de ejecución y son detectados por el linter).
2. **Estructurales / Tiempo de Ejecución (11 programas):** Errores como índices de qubits inválidos que hacen colapsar el programa.
3. **Semánticos Cuánticos (9 programas):** Defectos "silenciosos" (puertas incorrectas, desfase de *endianness*, etc.) donde el código se ejecuta sin arrojar excepciones pero genera distribuciones de salida erróneas.

Se evaluaron **18 modelos de lenguaje** (propietarios y de código abierto de entre 3B y 72B parámetros), comparando el desempeño sin guía (**B0**) contra la configuración fundamentada (**B2**) y variantes de mensajes de aviso (**B3/B4**).

3. Principales Hallazgos y Resultados

- **Detección de Defectos (RQ1):**

- La combinación del analizador estático con el detector LLM (RAG) incrementó la exhaustividad (*recall*) de **0.72 a 0.81**, manteniendo una precisión de **1.00** en el conjunto de prueba.
- El analizador estático no puede detectar defectos semánticos silenciosos, mientras que el detector LLM logró identificar varios de ellos.

- **Efecto Heterogéneo de la Reparación (RQ2):**
  - A nivel agregado, la fundamentación aumentó la precisión global de reparación en **16 puntos porcentuales** ($p < 0.001$).
  - **Sin embargo, el impacto varía drásticamente según el tipo de defecto:**
    - **Migración de API:** Ganancia masiva del **+50%** en la tasa de éxito (la información RAG aporta las firmas de API actualizadas).
    - **Estructurales:** Sin cambio neto (**0%**).
    - **Semánticos Silenciosos:** Caída del **-9%** en la precisión (y hasta caídas severas al evaluarlo en mutantes sintéticos).
- **Impacto según la Competencia del Modelo ("When Grounding Hurts"):**
  - En modelos avanzados (con mayor tasa base de ejecución de código), **agregar contexto externo empeoró el desempeño** para corregir defectos semánticos silenciosos (correlación parcial de -0.57).
  - Cuando el analizador estático no encuentra errores y reporta *"no hay defectos estructurales"*, los modelos avanzados tienden a ser distraídos por el contexto inútil o guiados a asumir erróneamente que el código no requiere corrección semántica, dejando el error intacto (fenómeno de distracción por contexto/siscofancia).
- **Efecto de los Avisos de Mitigación (RQ3):**
  - Proporcionar advertencias genéricas o nombradas de que el linter puede ser incompleto (variantes **B3/B4**) redujo algunas pérdidas en modelos fuertes (como Sonnet), pero deterioró el rendimiento en modelos más pequeños. Ninguna estrategia de aviso funcionó de forma óptima para todos los modelos.

4. Conclusión

El estudio demuestra un fenómeno clave: **el grounding no siempre ayuda**. Mientras que RAG y la validación por ejecución son altamente efectivos para corregir errores de

sintaxis y APIs desactualizadas, **pueden degradar la capacidad de razonamiento de los modelos más capaces al intentar reparar errores semánticos silenciosos**.

Este trabajo (Li et al., KDD 2026) presenta **QBench**, un benchmark de extremo a extremo (*end-to-end*) diseñado para evaluar la reparación automática de errores (*Automated Program Repair* - APR) en código y sistemas de software cuántico real.

1. El Benchmark: QBench

A diferencia del software clásico, el software cuántico presenta características particulares como **salidas probabilísticas**, **equivalencia basada en tolerancias numéricas** y **alta sensibilidad al entorno de ejecución y dependencias**. QBench aborda estos retos proporcionando un entorno estandarizado, ejecutable y reproducible para medir el desempeño de sistemas de reparación automática y agentes basados en LLMs.

2. Estructura y Dataset

QBench contiene **520 tareas de reparación** recopiladas de 129 repositorios de código abierto:

- **RepoTrack (264 instancias / 55 repositorios):** Se enfoca en defectos dentro de la infraestructura (compiladores, transpiladores, simuladores y SDKs). Requiere razonamiento a nivel de repositorio, modificaciones multi-archivo/módulo y manejo de dependencias.
- **ScriptTrack (256 instancias / 74 repositorios):** Se enfoca en programas cuánticos orientados al usuario final y flujos experimentales (algoritmos cuánticos, pipelines de autodiferenciación, scripts). Presenta correcciones más localizadas que requieren comprensión semántica del dominio.

Tipos de Oráculos de Correctitud:

- **Deterministas (56%–61%):** Pruebas unitarias tradicionales, contratos de API y validación exacta de salidas/excepciones.
- **Basados en Tolerancia (39%–44% en conjunto con probabilísticos):** Verificación de equivalencia numérica aproximada bajo umbrales aceptables ($ \delta$).
- **Probabilísticos:** Evaluación estadística de distribuciones de medición cuántica mediante ejecuciones repetidas (K repeticiones, S *shots*).

3. Evaluación y Modelos Evaluados

Se evaluaron **10 modelos de lenguaje** (propietarios y open-weight) junto con baselines tradicionales de reparación en un entorno containerizado y aislado:

Categoría	Modelos / Sistemas
LLMs Frontera	GPT-4o, Claude-3.5-Sonnet, Gemini-2.5-Pro
Especializados en Código	DeepSeek-Coder, Qwen2.5-Coder, Codestral-22B
Eficientes / Pequeños	GPT-4o-mini, DeepSeek-Coder-1.3B
Open-Weight	LLaMA-3.1-Instruct, DeepSeek-R1
Baselines Tradicionales	Plantillas heurísticas, APR basado en búsqueda (*Search-based APR*)

4. Principales Hallazgos

- **Efectividad General (Pass@1):**
    - Los LLMs superan con amplio margen a los baselines tradicionales (10.9%–17.1%).
    - **GPT-4o** obtuvo el mejor rendimiento global con **50.1%** (38.6% en RepoTrack y 61.7% en ScriptTrack), seguido de **Claude-3.5-Sonnet** (**48.8%**) y **Gemini-2.5-Pro** (**46.2%**).
    - **RepoTrack es sustancialmente más difícil:** Incluso los modelos más avanzados resuelven menos del 40% de los problemas a nivel de repositorio debido a la complejidad de integración, módulos cruzados y fragilidad del entorno.
- **Beneficio del Feedback Iterativo:**
    - La retroalimentación del oráculo (hasta 10 intentos) incrementa fuertemente la tasa de éxito.
    - En modelos frontera como GPT-4o, el feedback iterativo aportó un incremento de más de **17 puntos porcentuales** en RepoTrack al permitir corregir errores de compilación, dependencias y fallas parciales.
- **Dificultad según el Tipo de Oráculo:**
    - Los oráculos **deterministas** son los más fáciles de satisfacer.
    - Los oráculos **probabilísticos** representan la mayor barrera (con caídas de más de 20 puntos de éxito), ya que los modelos tienden a optimizar

estimaciones puntuales en lugar de resolver desajustes distribucionales subyacentes.

- **Fragilidad de los Parches Probabilísticos:**
    - Aprobar un oráculo probabilístico en una sola ejecución no garantiza que el parche sea semánticamente correcto. Al reevaluar repetidamente parches considerados "exitosos", los modelos frontera fallaron en cerca del **7%** de los ensayos, mientras que los métodos de búsqueda tradicional mostraron una varianza y fragilidad extremas.

5. Conclusión

QBench demuestra que la capacidad de razonamiento semántico y la escala del modelo son esenciales para reparar software cuántico. Sin embargo, la naturaleza estocástica de los oráculos, la sensibilidad a las versiones del entorno y los errores que cruzan múltiples capas de la pila cuántica siguen siendo los principales obstáculos para lograr una automatización confiable.

Este trabajo (Chen et al., 2026) presenta **HyQDB**, un marco de depuración en dos niveles (*tiered framework*) diseñado específicamente para detectar y reparar fallas **silenciosas** en programas cuánticos híbridos (como VQE, QAOA y QML) que combinan circuitos cuánticos con bucles de optimización clásicos.

1. El Problema y la Motivación

- **Fallas Silenciosas:** En flujos cuánticos híbridos, más del 75% de los errores reportados por expertos no hacen colapsar el programa (*crashes*), sino que se ejecutan correctamente y convergen a **resultados numéricamente incorrectos** (p. ej., parámetros desalineados, gradientes congelados o inversión de signos).
- **Sesgo de los Benchmarks Existentes:** Las herramientas y benchmarks previos basados en repositorios públicos de GitHub evalúan predominantemente errores que provocan excepciones o cuelgues, ignorando las fallas silenciosas que sufren los desarrolladores en la práctica.
- **Doble Naturaleza de las Fallas:** El estudio divide las fallas de programas híbridos en dos categorías principales:
    1. **Fallas Mecánicas:** Tienen firmas detectables mediante análisis sintáctico o trazado en tiempo de ejecución (p. ej., inicialización simétrica, tasas de aprendizaje inadecuadas o inconsistencias de puertas de hardware).
    2. **Fallas Conceptuales:** Discrepancias entre la intención semántica del programador y la implementación del algoritmo (p. ej., medir en la base equivocada o la falta de un operador en el ansatz).

2. La Solución: Arquitectura HyQDB

HyQDB utiliza un sistema guiado por **inyección de evidencia determinista** y un mecanismo de **escalamiento por silencio** (*escalation gate*):

- **Nivel 1: Analizadores Deterministas (Detectores):**
    - **Detector de Hardware:** Examina el código para verificar la compatibilidad de puertas nativas y topología del dispositivo destino.
    - **Detector de Optimización:** Combina escaneo del AST con profilado dinámico en tiempo de ejecución (interceptando llamadas a QNode en PennyLane). Detecta hiperparámetros fuera de rango, inicialización en puntos de silla y varianza nula o excesiva en las trayectorias de convergencia.
    - **Detector de Física:** Identifica el dominio científico (p. ej., química cuántica) y valida que no se violen invariantes físicas (conservación de número de partículas, simetría de espín o conteo de *shots* crítico).

    - *Acción:* Si se detecta una falla, la evidencia se estructurariza en formato JSON y se inyecta directamente al prompt de reparación del LLM.

- **Nivel 2: Reconstrucción de Intención (*Intent Reconstruction*):**
    - Si todos los detectores permanecen en **silencio** (sin evidencia encontrada), HyQDB interpreta esa ausencia como una señal de que se trata de una **falla conceptual**.
    - **Escalamiento:** Un primer llamado al LLM infiere la intención del algoritmo analizando nombres de variables, funciones y comentarios. Un segundo llamado reconcilia el código contra dicha especificación inferida para generar la corrección.

3. El Benchmark: QFaultBench

Para evaluar el sistema, los autores crearon **QFaultBench**, un conjunto de **62 tareas de reparación** distribuidas en 10 algoritmos reales en PennyLane (usando Autograd o JAX):

- **Set de Desarrollo (34 tareas):** Inyecciones de errores generadas mediante LLM para calibración.
- **Set de Evaluación Retenido / Held-Out (28 tareas):** Construido de forma independiente por **expertos humanos** para comprobar la capacidad de generalización del sistema.
- **Criterio de Evaluación:** Todos los programas se ejecutan sin arrojar excepciones y su corrección se verifica mediante *asserts* ocultos en tiempo de ejecución.

4. Principales Hallazgos y Resultados

- **Aumento en la Tasa de Reparación:**
    - En el conjunto de prueba humano (*held-out*), un LLM base (DeepSeek-v4-flash) logra reparar únicamente el **45%** de los errores.
    - Al agregar la evidencia de los detectores deterministas, la tasa sube al **56%**.
    - Al integrar el flujo completo de dos niveles con reconstrucción de intención, alcanza un **75%** de éxito (**+30 puntos porcentuales** de mejora).
- **Efectividad del Mecanismo de Escalamiento (*Gating*):**

    - El mecanismo encamina correctamente entre el **76% y el 88%** de los errores conceptuales hacia la fase de intención.
    - Desactivar el escalamiento y forzar la fase de intención en todos los programas reduce la precisión de reparación de fallas mecánicas en un **19%**, ya que se pierde la localización exacta que aportan las herramientas deterministas.

- **Generalización en Diversos Modelos:**
    - La mejora se mantiene consistente en distintos modelos base (incluyendo DeepSeek-v4-flash, Gemini-3.6-flash y GPT-5-mini), alcanzando un techo de rendimiento similar.

- **Eficiencia en el Uso de Tokens:**
    - Para fallas mecánicas, proveer evidencia determinista directa **reduce el consumo de tokens** respecto al baseline, al evitar intentos y errores fallidos.
    - Para fallas conceptuales, la reconstrucción de intención incrementa el uso de tokens entre **2x y 4x** debido a las llamadas adicionales de razonamiento semántico.

5. Conclusión

HyQDB demuestra que la depuración efectiva de flujos cuánticos híbridos requiere tratar las fallas según su naturaleza. La combinación de análisis dinámico determinista para errores mecánicos con la reconstrucción de intención semántica para errores conceptuales es fundamental para automatizar la corrección de fallas silenciosas en programas cuánticos.

Este trabajo (Li et al., 2026; publicado en *ACL 2026 Findings*) aborda la falta de confiabilidad en las explicaciones generadas por Modelos de Lenguaje de Gran Escala (LLM) durante la reparación automática de programas cuánticos (*Automated Program Repair* - APR).

1. El Problema y la Motivación

- **Explicaciones Inverificables:** Aunque los LLM pueden sugerir parches y generar explicaciones en lenguaje natural para justificar sus correcciones, estas explicaciones son narrativas *post-hoc* que no se pueden verificar formalmente.
- **Sensibilidad del Software Cuántico:** Pequeños cambios sintácticos en circuitos cuánticos pueden alterar drásticamente su comportamiento semántico o la fidelidad del estado. Las explicaciones plausibles pero falsas resultan peligrosas, ya que un parche puede parecer correcto narrativamente pero ser semánticamente erróneo.

2. La Propuesta: Explicaciones Ejecutables mediante Trazas de Prueba

Los autores proponen transformar las explicaciones en **objetos ejecutables y verificables mecánicamente**. En lugar de generar texto libre, el sistema exige que las explicaciones se compilen en **trazas de prueba** formales compuestas por reglas de transformación verificables.

3. Metodología del Marco de Trabajo

El proceso opera en tres etapas principales:

1. **Generación de Candidatos con Razonamientos Estructurados:** El LLM genera candidatos de parche acompañados de *razonamientos estructurados* bajo un esquema restringido (especificando conmutación de puertas, sustitución, descomposición y justificación semántica).
2. **Compilación a Trazas de Prueba Ejecutables:** Los razonamientos estructurados se compilan de forma determinista en trazas de prueba ejecutablesCompuestas por reglas de transformación formales.
3. **Verificación Formal Multi-Backend y Selección:** Las trazas de prueba se evalúan mediante múltiples motores de verificación complementarios:
    - **Comprobador de Equivalencia de Circuitos:** Para subcircuitos unitarios.
    - **Verificador de Cálculo ZX (PyZX):** Para normalización y reglas de reescritura diagramática.

- **Razonamiento basado en Estabilizadores:** Para circuitos dominados por el grupo de Clifford.
- **Simulación Cuántica:** Para estimación de fidelidad cuando la equivalencia exacta es intratable.

Solo se aceptan los parches cuyas explicaciones completas son **aprobadas en su totalidad** por los verificadores formales. El parche final se selecciona minimizando la distancia de edición del código y la brevedad de la traza de prueba.

4. Experimentos y Resultados Principales

- **Benchmark:** Evaluado sobre **QASMBench** (500 circuitos de 2 a 130 qubits), generando **5,000 circuitos defectuosos** mediante 5 operadores de mutación (eliminación, inserción y reordenamiento de puertas, perturbación de parámetros y mutación de índices de qubits).
- **Rendimiento General:**
  - **Tasa de Éxito de Reparación: 76.1%** (superando a baselines de LLM puros y métodos basados en búsqueda/síntesis).
  - **Precisión Semántica: 96.1%** de los parches aceptados fueron semánticamente equivalentes al original (frente al 65.9%–68.4% de los baselines de LLM tradicionales).
  - **Fidelidad de la Explicación (*Explanation Faithfulness*):** Alcanzó un **94.8%** de aceptación por los verificadores formales (en comparación con solo el 18.6% para explicaciones en lenguaje natural).
  - **Distancia de Edición:** Promedio de **0.28**, produciendo las modificaciones más minimalistas.
- **Análisis de Ablación y Escalabilidad:**
  - El motor de reescritura **ZX-calculus** demostró ser el componente más crítico; removerlo causó la mayor caída en la fidelidad de las explicaciones.
  - El sistema escala eficazmente hasta circuitos de tamaño mediano (~14 qubits) con baja sobrecarga de tiempo, y mantiene una fidelidad superior al **88%** incluso en circuitos complejos de más de 65 qubits.
- **Independencia del Modelo Base:**
  - Evaluado en GPT-4o-mini, Claude-3.5-Sonnet, DeepSeek-R1 y Gemini-2.5-Pro. La capa de verificación mantuvo una alta precisión

semántica en todos los modelos, filtrando correcciones espurias independientemente del tamaño del LLM.

5. Conclusión

El estudio demuestra que es posible unir la capacidad generativa de los LLMs con la solidez de la verificación formal para el software cuántico. Reemplazar las explicaciones narrativas por trazas de prueba verificables garantiza que las correcciones aceptadas sean no solo plausibles, sino **comprobablemente correctas**.

Este trabajo (Pham et al., 2026) presenta **QBugLM**, un marco de evaluación multi-agente para automatizar el flujo de depuración (*debugging*) de software cuántico en programas escritos en OpenQASM 3.0 independiente del kit de desarrollo (SDK).

1. La Arquitectura QBugLM

QBugLM integra cuatro componentes principales para ejecutar y validar el proceso de detección, reparación y verificación:

- **QBugGen:** Herramienta de mutación que inyecta fallas de forma sistemática en circuitos cuánticos válidos. Clasifica las fallas en cuatro categorías:
    1. *Errores de sintaxis obsoleta* (sintaxis no compatible).
    2. *Errores estructurales* (p. ej., qubits de control y objetivo idénticos).
    3. *Uso redundante de puertas* (duplicación de puertas autoinversas).
    4. *Desviación semántica* (sustitución, eliminación, adición de puertas o alteración de ángulos de rotación).
- **QBugFind:** Agente LLM encargado de analizar el código defectuoso y generar un informe estructurado que identifica la ubicación y la categoría del error.
- **QBugFix:** Agente LLM independiente que recibe el código y el informe de QBugFind para generar la versión corregida.
- **QBugCheck:** Validador determinista que ejecuta el circuito corregido y el de referencia en un simulador sin ruido (Qiskit Aer). Confirma la validez mediante la **distancia de variación total** ($\delta \le 0.05$) de la distribución de mediciones y la equivalencia en el conteo de puertas transpiladas.

2. Configuración Experimental

- **Modelos Evaluados: Claude Sonnet 4.6** (propietario) y **Qwen3 Coder Next** (código abierto).
- **Estrategias de Prompting:** *Structured* (estructurado), *Chain-of-Thought* (CoT) y *ReAct*.
- **Conjunto de Datos:** Circuitos de 5 qubits del benchmark MQT Bench (Deutsch-Jozsa, Grover, Bernstein-Vazirani, GHZ y W-state).

3. Principales Hallazgos

- **Estrategias de Prompting (RQ1):**

- El estilo de interacción **estructurado** (*Structured*) superó consistentemente a *Chain-of-Thought* (CoT) y *ReAct*.

- En modelos con capacidad nativa de razonamiento, forzar esquemas rígidos de pensamiento (CoT/ReAct) degradó el rendimiento en lugar de mejorarlo bajo restricciones de recursos.

- **Capacidad de Detección y Reparación (RQ2):**

  - La depuración en un solo intento (*0 retries*) es altamente ineficaz (Pass@1 < 25% en la mayoría de categorías, excepto en errores estructurales con un 60%).

  - **El feedback iterativo es el factor decisivo:** Permitir un único reintento elevó la tasa Pass@1 de <25% a >80%. Con dos reintentos, ambos modelos recuperaron un rendimiento cercano o igual al 100% en casi todas las categorías.

- **Eficiencia de Costos y Tiempo (RQ3):**

  - El modelo de código abierto **Qwen3 Coder Next** demostró ser sustancialmente más eficiente en costos y velocidad: redujo los costos financieros entre **4.8x y 9.1x** y fue de **1.5x a 4.6x más rápido** que Claude Sonnet 4.6 en errores estructurales, sintácticos y de redundancia de puertas.

  - **Excepción semántica:** En las desviaciones semánticas complejas, Claude Sonnet 4.6 alcanzó un 100% de Pass@1 frente al 92% de Qwen3 Coder Next, justificando su mayor costo sólo para esa categoría específica.

4. Conclusión

El estudio demuestra que los sistemas multi-agente respaldados por un bucle de retroalimentación iterativo permiten depurar código cuántico de manera altamente efectiva. Además, subraya que para tareas de depuración en lenguajes como OpenQASM 3.0, un prompting estructurado simple combinado con modelos abiertos especializados resulta más económico y rápido que utilizar arquitecturas propietarias complejas.

Este trabajo (d'Aloisio et al., ESEM 2024) presenta el primer estudio empírico que evalúa la capacidad de los Modelos de Lenguaje de Gran Escala (LLM) para explicar algoritmos cuánticos escritos en OpenQASM 3, con el objetivo de reducir la barrera de entrada para los ingenieros de software clásicos.

1. Configuración del Estudio

- **Modelos evaluados:** GPT-3.5 (turbo 16k), LLaMA2 (vía Ollama) y TinyLLaMA.
- **Benchmark:** 7 algoritmos cuánticos representativos de MQT Bench (*Amplitude Estimation*, *Deutsch-Jozsa*, *Grover*, *QFT*, *QFT con entrelazamiento*, *QPE* y *Quantum Walk*).
- **Estilos de Prompts:**
    - **Sin Contexto (*Non Context-Aware*):** Solo se proporciona el código OpenQASM pidiendo una explicación de alto nivel.
    - **Con Contexto (*Context-Aware*):** Se incluye además el nombre del algoritmo y el número de qubits empleados.
- **Evaluación Humana:** 4 evaluadores independientes (ingenieros de software con distinto grado de familiaridad en computación cuántica) calificaron las explicaciones en una escala Likert de 1 a 5 (donde 1 es totalmente errónea y 5 es de alta calidad).

2. Principales Hallazgos y Respuestas a las Preguntas de Investigación

- **Calidad de Generación desde Cero (RQ1):**
    - **LLaMA2** obtuvo el mejor desempeño general para generar explicaciones desde cero, alcanzando una puntuación media de **2.78** y una mediana de **2.75**.
    - **GPT-3.5** tuvo un desempeño cercano con una media y mediana de **2.50**.
    - **TinyLLaMA** mostró un rendimiento marcadamente inferior (media de **1.87**), produciendo una alta proporción de explicaciones sistemáticamente erróneas (calificadas con 1.0).
- **Impacto del Contexto en el Prompt (RQ2):**
    - Agregar información básica sobre el algoritmo (nombre y número de qubits) incrementó de forma estadísticamente significativa la calidad de las explicaciones en **todos** los modelos evaluados ($p < 0.001$).

    - La proporción de explicaciones con puntuaciones bajas (1 o 2) se redujo drásticamente en GPT-3.5 y LLaMA2 al usar prompts con contexto.

- **Consistencia de las Explicaciones (RQ3):**
    - Al repetir la generación durante 3 rondas independientes, se observó alta **estabilidad cualitativa** (sin diferencias estadísticas significativas entre rondas en la prueba de Kruskal-Wallis).
    - En términos de **similitud sintáctica** (medida mediante similitud del coseno), GPT-3.5 y LLaMA2 mantuvieron puntuaciones elevadas de consistencia (mediana entre 0.7 y 0.8), mientras que TinyLLaMA presentó mayor variabilidad estocástica.
- **Capacidad para Refinar Explicaciones Existentes (RQ4):**
    - Cuando se les pidió mejorar y hacer más informativas descripciones breves ya existentes, **GPT-3.5** demostró ser el mejor modelo, alcanzando una puntuación media de **3.54** (mediana de **4.50**), superando a LLaMA2 (2.39) y TinyLLaMA (1.89).

3. Conclusión y Trabajo Futuro

El estudio demuestra que los LLM como LLaMA2 y GPT-3.5 ofrecen un potencial prometedor para apoyar la comprensión de programas cuánticos. Sin embargo, ningún modelo logró un promedio superior a 4/5 en la generación desde cero sin contexto previo, lo que confirma que el código cuántico (basado en circuitos lógicos y puertas) plantea retos particulares respecto al código clásico. Se señalan como líneas futuras la optimización de prompts, el filtrado/parsing estructurado de respuestas y el uso de guías de evaluación más cuantitativas.

Este artículo (Khan et al., 2026) presenta una metodología automatizada para categorizar las preguntas y discusiones de los desarrolladores de Ingeniería de Software Cuántico (*Quantum Software Engineering* - QSE) en comunidades en línea como Stack Overflow, utilizando modelos de aprendizaje por transferencia (*Transfer Learning*) e Inteligencia Artificial Explicable (XAI).

1. Dataset y Creación del Ground Truth

- **Conjunto de Datos:** Se recopilaron **2,829 preguntas** de desarrolladores extraídas de cuatro plataformas Q&A: Stack Overflow (SO), Quantum Computing Stack Exchange (QCSE), Computer Science Stack Exchange (CSSE) y Artificial Intelligence Stack Exchange (AISE).

- **Anotación Colaborativa con ChatGPT:** Para construir el dataset etiquetado (*ground truth*), se utilizó una metodología cualitativa combinando anotaciones humanas y ChatGPT-3.5.

- **Resolución de Conflictos mediante Negociación:** Se identificaron 282 desacuerdos (10%) entre los evaluadores humanos y ChatGPT. Tras una segunda ronda de negociación y justificación conversacional, se logró un consenso total para construir un dataset libre de conflictos.

2. Taxonomía de Desafíos en QSE

A través de *Grounded Theory* y análisis de contenido, las discusiones se clasificaron en seis categorías de desafíos principales:

1. **Errors (28.80% - 815 publicaciones):** Depuración, resolución de problemas técnicos, fallos en simuladores y problemas de compatibilidad de hardware.

2. **Conceptual (21.56% - 610 publicaciones):** Comprensión de conceptos básicos, limitaciones y fundamentos de las APIs cuánticas.

3. **Tooling (21.06% - 596 publicaciones):** Uso y consulta sobre herramientas de software, simuladores y entornos como IBM Quantum Experience o QASM.

4. **Theoretical (14.66% - 415 publicaciones):** Fundamentos matemáticos, física subyacente, algoritmos (como Shor o Grover) y complejidad computacional.

5. **API Usage (8.02% - 227 publicaciones):** Integración de APIs cuánticas (Q#, Qiskit, pyQuil) y equivalencias sintácticas con lenguajes clásicos.

6. **Learning (5.86% - 166 publicaciones):** Solicitud de recursos educativos, libros, cursos y tutoriales para principiantes.

3. Clasificación Automatizada con Transfer Learning

El estudio comparó modelos de Transfer Learning (basados en Transformers) frente a clasificadores tradicionales de Aprendizaje Profundo (DL) y Aprendizaje Automático (ML):

- **Desempeño de Transformers:** Los modelos **BERT** y **DistilBERT** alcanzaron una **precisión promedio del 95%** utilizando los datos reales sin necesidad de generación sintética o aumentación de datos.
- **Comparación con Aprendizaje Profundo Tradicional:**
    - **CNN:** Logró un 86% de precisión *con* aumentación de datos, pero cayó al 51% *sin* aumentación.
    - **MLP (Perceptrón Multicapa):** Alcanzó un 67% *con* aumentación de datos y un 54% *sin* ella.
- **Ventaja Clave:** Los modelos de Transfer Learning aprovechan representaciones contextuales bidireccionales preentrenadas, lo que les permite manejar vocabulario técnico y abstracto en entornos de pocos recursos sin requerir datos sintéticos.

4. Explicabilidad del Modelo mediante SHAP (XAI)

Para resolver la naturaleza de "caja negra" del modelo BERT, se aplicó la técnica **SHAP** (*SHapley Additive exPlanations*):

- **Interpretabilidad Global:** Reveló los términos lingüísticos que más influyen en la clasificación general (ej. tokens como *"Programmer"*, *"Register"*, *"Command"*, *"Function"* y *"Physics"*).
- **Interpretabilidad Local (Force Plots):** Evaluó predicciones individuales para mostrar exactamente qué palabras clave aumentaban o disminuían la confianza del modelo hacia una categoría (por ejemplo, palabras como *"error"*, *"dotnet restore"* y *"Microsoft"* orientaron la clasificación directamente hacia la categoría *Errors*).

5. Conclusión e Implicaciones

El trabajo demuestra que el uso de Transfer Learning junto con SHAP proporciona una solución precisa y transparente para categorizar automáticamente las dudas de los desarrolladores. Esta categorización permite a los proveedores de software cuántico (como IBM, Microsoft o Rigetti) y a los administradores de foros organizar mejor el conocimiento, detectar las principales fallas del ecosistema y mejorar la documentación de sus APIs.